

Task (i): Facts, Entities, Predicates, Variables and Constants

1.1 Entities in the Domain

An entity is any real-world object the knowledge base reasons about. In this scenario the entities are:

- Ravi the patient under diagnosis (a specific individual).
- Symptoms fever, cough, body pain (observable clinical signs).
- Diseases / conditions flu, viral infection (intermediate diagnostic hypotheses).
- Outcome the requirement of medical attention (the action recommended by the system).

1.2 Facts Extracted from the Scenario

Facts are ground statements sentences with no variables that are asserted to be true about a specific individual. The three facts stated in the scenario are:

1. Ravi has fever.
2. Ravi has cough.
3. Ravi has body pain.

1.3 Rules (General Knowledge) Extracted from the Scenario

Rules are general, patient-independent statements that hold for any patient x. They form the inference engine's knowledge base:

1. If a patient has fever and cough, the patient may have flu.
2. If a patient has flu and body pain, the patient may have a viral infection.
3. If a patient has a viral infection, the patient requires medical attention.

1.4 Predicates

A predicate is a function that maps an object (or objects) to a truth value. Each predicate below takes one argument the patient and returns True or False depending on whether the described property holds for that patient.

Predicate	Meaning	Arity
Fever(x)	x has fever	1
Cough(x)	x has cough	1
BodyPain(x)	x has body pain	1
Flu(x)	x has flu	1
ViralInfection(x)	x has a viral infection	1
RequiresMedicalAttention(x)	x requires medical attention	1

1.5 Variables and Constants

Type	Symbol(s)	Explanation
Variable	x	A placeholder standing for an arbitrary, unspecified patient. Used only inside the general rules, which must hold for every patient.
Constant	Ravi	A specific, named individual in the domain. Constants are substituted for variables once a rule is applied to a particular patient.

Together, the facts (§1.2), the rules (§1.3) and the predicate vocabulary (§1.4) constitute the complete knowledge base KB that will be formalised symbolically in Tasks (ii) and (iii), and mechanically reasoned over in Tasks (iv)–(vi).

Task (ii): Propositional Logic Representation

Propositional logic represents each statement as an indivisible atomic proposition – it cannot express ‘for every patient’ because it has no variables or quantifiers. Since the scenario concerns one specific patient (Ravi), each predicate instantiated on Ravi is assigned its own propositional symbol:

Symbol	Proposition
P	Ravi has fever
Q	Ravi has cough
R	Ravi has body pain
S	Ravi has flu
T	Ravi has a viral infection
U	Ravi requires medical attention

2.1 Known Facts (Asserted True)

$P = \text{True}$, $Q = \text{True}$, $R = \text{True}$

2.2 Rules as Propositional Implications

Each English rule is translated using the logical connectives AND ($\wedge$), OR ($\vee$), NOT ($\neg$) and implication ($\rightarrow$):

R1: $(P \wedge Q) \rightarrow S$ "if fever and cough, then flu"

R2: $(S \wedge R) \rightarrow T$ "if flu and body pain, then viral infection"

R3: $T \rightarrow U$ "if viral infection, then requires medical attention"

2.3 Complete Propositional Knowledge Base

$KB = \{ P, Q, R, (P \wedge Q) \rightarrow S, (S \wedge R) \rightarrow T, T \rightarrow U \}$

Query: Is T ("Ravi has a viral infection") entailed by KB , i.e. does $KB \models T$ hold? A quick truth-table / forward-substitution check confirms it: since P and Q are both True, $R1$'s antecedent $(P \wedge Q)$ is True, so S must be True; since S and R are both True, $R2$'s antecedent $(S \wedge R)$ is True, so T must be True. This informal check is made rigorous through resolution in Task (v).

Limitation to note: propositional logic can only encode facts about Ravi individually. If the hospital had a second patient, Priya, an entirely new set of propositions (P' , Q' , R' , ...) and rules would be required, because propositional symbols cannot be parameterised. This motivates moving to first-order logic in Task (iii).

Task (iii): First-Order Logic (FOL) Representation

FOL overcomes the limitation above by introducing variables, predicates and quantifiers, so a single rule applies uniformly to every patient in the domain. The universal quantifier $\forall$ ("for all") is used because each diagnostic rule must hold for any patient x , not just Ravi.

3.1 General Rules (Universally Quantified)

Rule 1: $\forall x (\text{Fever}(x) \wedge \text{Cough}(x)) \rightarrow \text{Flu}(x)$

Rule 2: $\forall x (\text{Flu}(x) \wedge \text{BodyPain}(x)) \rightarrow \text{ViralInfection}(x)$

Rule 3: $\forall x \text{ ViralInfection}(x) \rightarrow \text{RequiresMedicalAttention}(x)$

Reading Rule 1 aloud: "For every x , if x has fever and x has cough, then x has flu." The variable x is bound by $\forall$ and ranges over the set of all patients in the domain of discourse.

3.2 Ground Facts about Ravi

$\text{Fever}(\text{Ravi}) \quad \text{Cough}(\text{Ravi}) \quad \text{BodyPain}(\text{Ravi})$

These are ground atoms they contain the constant Ravi in place of the variable x , and therefore have no quantifier; a ground sentence is either true or false outright, not "for some / for all".

3.3 The Query

Query: $\text{ViralInfection}(\text{Ravi}) ?$ (and, by extension, $\text{RequiresMedicalAttention}(\text{Ravi}) ?$)

An existential quantifier ($\exists$, "there exists") is not required anywhere in this particular knowledge base because every rule is a general law applying to all patients, and every fact concerns one already-named

individual, Ravi. ($\exists$ would appear, for example, in a sentence such as “Some patient in the ward has a viral infection”: $\exists x \text{ ViralInfection}(x)$.)

The complete FOL knowledge base is therefore:

$\text{KB} = \{ \forall x(\text{Fever}(x) \wedge \text{Cough}(x)) \rightarrow \text{Flu}(x), \quad \forall x(\text{Flu}(x) \wedge \text{BodyPain}(x)) \rightarrow \text{ViralInfection}(x), \quad \forall x$
 $\text{ViralInfection}(x) \rightarrow \text{RequiresMedicalAttention}(x), \text{Fever}(\text{Ravi}), \text{Cough}(\text{Ravi}), \text{BodyPain}(\text{Ravi}) \}$

Task (iv): Unification

Unification is the algorithm that finds a substitution θ which, when applied to two logical expressions, makes them syntactically identical. It is the mechanism that lets a general rule (containing a variable) be matched against a specific fact (containing a constant), which is exactly what is needed to fire Rule 1 for the patient Ravi.

4.1 Unification Example 1 — Rule 1's conclusion with the query pattern

Consider the consequent of Rule 1, $\text{Flu}(x)$, and the constant term $\text{Flu}(\text{Ravi})$ that we need to derive. Unify:

$\text{Flu}(x)$ and $\text{Flu}(\text{Ravi})$

Step-by-step application of the unification algorithm:

1. Predicate symbols match: both are “Flu”, and both have arity 1. ✓
2. Compare argument 1: x (a variable) vs Ravi (a constant).
3. Since one of the two terms is a variable, bind it to the other term: $\theta = \{ x / \text{Ravi} \}$.
4. Apply θ to both expressions: $\text{Flu}(x)\theta = \text{Flu}(\text{Ravi})$ and $\text{Flu}(\text{Ravi})\theta = \text{Flu}(\text{Ravi})$ — identical. Unification succeeds.
5. Most General Unifier: $\text{MGU} = \{ x / \text{Ravi} \}$

4.2 Unification Example 2 matching the antecedent of Rule 1 against the facts

Rule 1's antecedent is the conjunct $\text{Fever}(x) \wedge \text{Cough}(x)$. To fire the rule for Ravi, each conjunct must unify with a known fact:

$\text{Fever}(x)$ unify with $\text{Fever}(\text{Ravi}) \rightarrow \theta_1 = \{ x / \text{Ravi} \}$

$\text{Cough}(x)$ unify with $\text{Cough}(\text{Ravi}) \rightarrow \theta_2 = \{ x / \text{Ravi} \}$

Both unifications yield the same binding for x , so they are consistent; the single combined substitution $\theta = \{ x / \text{Ravi} \}$ satisfies the entire antecedent simultaneously, which is a precondition for the rule to fire (see Task v).

4.3 Unification Example 3 chaining Rule 1 into Rule 2

Rule 2 is $\forall y (\text{Flu}(y) \wedge \text{BodyPain}(y)) \rightarrow \text{ViralInfection}(y)$. Note the variable has been renamed from x to y (“standardising apart”) a mandatory pre-processing step in resolution so that two different rules never

accidentally share a variable name. We now unify the derived fact $\text{Flu}(\text{Ravi})$ (obtained in §4.1) with Rule 2's first conjunct $\text{Flu}(y)$:

$\text{Flu}(\text{Ravi})$ and $\text{Flu}(y) \rightarrow \text{MGU} = \{ y / \text{Ravi} \}$

Applying this MGU throughout Rule 2 grounds it completely for Ravi, allowing $\text{BodyPain}(y)$ to be checked next as $\text{BodyPain}(\text{Ravi})$ which is already a known fact so Rule 2 can fire and derive $\text{ViralInfection}(\text{Ravi})$.

4.4 Summary of Substitutions Used

Unified pair	Most General Unifier (MGU)
$\text{Fever}(x)$, $\text{Fever}(\text{Ravi})$	$\{ x / \text{Ravi} \}$
$\text{Cough}(x)$, $\text{Cough}(\text{Ravi})$	$\{ x / \text{Ravi} \}$
$\text{Flu}(x)$, $\text{Flu}(\text{Ravi})$	$\{ x / \text{Ravi} \}$
$\text{Flu}(y)$, $\text{Flu}(\text{Ravi})$	$\{ y / \text{Ravi} \}$
$\text{BodyPain}(y)$, $\text{BodyPain}(\text{Ravi})$	$\{ y / \text{Ravi} \}$
$\text{ViralInfection}(z)$, $\text{ViralInfection}(\text{Ravi})$	$\{ z / \text{Ravi} \}$

Task (v): Resolution Proving $\text{ViralInfection}(\text{Ravi})$

Resolution is a sound and complete inference rule for clausal (CNF) sentences. Because every rule here is a single Horn implication, converting to clause form is direct: an implication $(A \wedge B) \rightarrow C$ is logically equivalent to the clause $\neg A \vee \neg B \vee C$ (obtained by $\neg(A \wedge B) \vee C$, then De Morgan's law).

5.1 Converting the Knowledge Base to Clause Form

ID	Clause (disjunction of literals)	Origin
C1	$\neg \text{Fever}(x) \vee \neg \text{Cough}(x) \vee \text{Flu}(x)$	Rule 1
C2	$\neg \text{Flu}(x) \vee \neg \text{BodyPain}(x) \vee \text{ViralInfection}(x)$	Rule 2
C3	$\neg \text{ViralInfection}(x) \vee \text{RequiresMedicalAttention}(x)$	Rule 3
C4	$\text{Fever}(\text{Ravi})$	Fact
C5	$\text{Cough}(\text{Ravi})$	Fact
C6	$\text{BodyPain}(\text{Ravi})$	Fact

5.2 Proof by Refutation

Resolution proves $KB \models \alpha$ by showing that $KB \cup \{\neg\alpha\}$ is unsatisfiable i.e. resolving clauses together eventually derives the empty clause (a contradiction, denoted NIL). We negate the query and add it to the clause set:

Negated Goal, C7: $\neg\text{ViralInfection}(\text{Ravi})$

Resolution proceeds by repeatedly selecting two clauses that contain complementary literals (one is the negation of the other, after unification), and resolving them into a new clause containing everything else from both parents:

1. Resolve $C1 = \neg\text{Fever}(x) \vee \neg\text{Cough}(x) \vee \text{Flu}(x)$ with $C4 = \text{Fever}(\text{Ravi})$.
2. Unify $\text{Fever}(x)$ with $\text{Fever}(\text{Ravi})$: $\text{MGU} = \{x/\text{Ravi}\}$.
3. The complementary pair $\neg\text{Fever}(\text{Ravi}) / \text{Fever}(\text{Ravi})$ cancels.
4. $\Rightarrow C8: \neg\text{Cough}(\text{Ravi}) \vee \text{Flu}(\text{Ravi})$
5. Resolve $C8 = \neg\text{Cough}(\text{Ravi}) \vee \text{Flu}(\text{Ravi})$ with $C5 = \text{Cough}(\text{Ravi})$.
6. Complementary pair $\neg\text{Cough}(\text{Ravi}) / \text{Cough}(\text{Ravi})$ cancels. $\Rightarrow C9: \text{Flu}(\text{Ravi})$
7. Resolve $C2 = \neg\text{Flu}(x) \vee \neg\text{BodyPain}(x) \vee \text{ViralInfection}(x)$ with $C9 = \text{Flu}(\text{Ravi})$.
8. Unify $\text{Flu}(x)$ with $\text{Flu}(\text{Ravi})$: $\text{MGU} = \{x/\text{Ravi}\}$.
9. Cancel $\neg\text{Flu}(\text{Ravi}) / \text{Flu}(\text{Ravi})$. $\Rightarrow C10: \neg\text{BodyPain}(\text{Ravi}) \vee \text{ViralInfection}(\text{Ravi})$
10. Resolve $C10 = \neg\text{BodyPain}(\text{Ravi}) \vee \text{ViralInfection}(\text{Ravi})$ with $C6 = \text{BodyPain}(\text{Ravi})$.
11. Cancel $\neg\text{BodyPain}(\text{Ravi}) / \text{BodyPain}(\text{Ravi})$. $\Rightarrow C11: \text{ViralInfection}(\text{Ravi})$
12. Resolve $C11 = \text{ViralInfection}(\text{Ravi})$ with the negated goal $C7 = \neg\text{ViralInfection}(\text{Ravi})$.
13. These are exact complements with no unification needed beyond identity. $\Rightarrow C12: \text{NIL}$ (the empty clause)

Deriving NIL means the assumption $\neg\text{ViralInfection}(\text{Ravi})$ is inconsistent with the knowledge base. Therefore the original, un-negated query is proved:

$\therefore KB \models \text{ViralInfection}(\text{Ravi})$ Ravi has a viral infection. ■

5.3 Extending the Proof to RequiresMedicalAttention(Ravi)

Clause $C11$ ($\text{ViralInfection}(\text{Ravi})$) can be resolved once more with $C3 = \neg\text{ViralInfection}(x) \vee \text{RequiresMedicalAttention}(x)$ using $\text{MGU} \{x/\text{Ravi}\}$, cancelling the complementary literals:

Resolve $C11$ with $C3 \Rightarrow C13: \text{RequiresMedicalAttention}(\text{Ravi})$

This single additional resolution step establishes the system's ultimate diagnostic recommendation directly from the proven viral-infection fact.

Task (vi): Complete Inference Sequence, Mapping Diagram and Final Conclusion

6.1 Forward-Chaining View of the Same Derivation

Task (v) proved the goal backward, by refutation. The identical conclusion is reached more intuitively by forward chaining starting from the known facts and firing rules whenever their antecedents become satisfied:

Step	Known before step	Rule fired (with MGU)	New fact derived
1	Fever(Ravi), Cough(Ravi)	Rule 1, {x/Ravi}	Flu(Ravi)
2	Flu(Ravi), BodyPain(Ravi)	Rule 2, {x/Ravi}	ViralInfection(Ravi)
3	ViralInfection(Ravi)	Rule 3, {x/Ravi}	RequiresMedicalAttention(Ravi)

6.2 Mapping / Inference-Flow Diagram

The diagram below traces symptoms through each rule to the final recommendation. Each arrow represents one rule firing, labelled with the MGU that grounded it for the constant Ravi.

Stage	Nodes (state of knowledge)	→ Fires rule →
0. Observed symptoms	Fever(Ravi) Cough(Ravi) BodyPain(Ravi)	Rule 1 {x/Ravi}
1. First diagnosis	Flu(Ravi) (+ BodyPain(Ravi) carried forward)	Rule 2 {x/Ravi}
2. Second diagnosis	ViralInfection(Ravi)	Rule 3 {x/Ravi}
3. Final recommendation	RequiresMedicalAttention(Ravi)	(goal reached)

Read top to bottom, this table is equivalent to the linear chain:

{Fever(Ravi), Cough(Ravi)} →[R1]→ Flu(Ravi) →[R2, with BodyPain(Ravi)]→ ViralInfection(Ravi) →[R3]→ RequiresMedicalAttention(Ravi)

6.3 Final Conclusion

Both the propositional-logic entailment check (Task ii), the resolution refutation proof (Task v), and the forward-chaining trace (§6.1) agree on the same result. Formally:

KB ⊨ ViralInfection(Ravi) and KB ⊨ RequiresMedicalAttention(Ravi)